

Section - 1

Basic Inheritance(Single Inheritance):

1. Create a class Animal with properties like name and age.
Create a subclass Dog that inherits from Animal and adds a property breed. Demonstrate the use of constructors in both the Animal and Dog classes.

```
public class workshop4 {  
    Run | Debug  
    public static void main(String[] args) {  
        Dog myDog = new Dog(name:"jenish", age:3, breed:"husky");  
  
        System.out.println("Name: " + myDog.getName());  
        System.out.println("Age: " + myDog.getAge());  
        System.out.println("Breed: " + myDog.getBreed());  
    }  
}  
  
class Animal {  
    private String name;  
    private int age;  
  
    public Animal(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}  
  
class Dog extends Animal {  
    private String breed;  
  
    public Dog(String name, int age, String breed) {  
        super(name, age);  
        this.breed = breed;  
    }  
}
```

```
}  
  
    public String getBreed() {  
        return breed;  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4.java  
Name: jenish  
Age: 4  
Breed: husky  
PS C:\Users\sigma\OneDrive\Desktop\java> 
```

Method Overriding:

2. Write a Java program to create a class called Shape with methods called `getPerimeter()` and `getArea()`. Create a subclass called Circle that overrides the `getPerimeter()` and `getArea()` methods to calculate the area and perimeter of a circle.

```
import java.lang.Math;
class Shape {
    public double getPerimeter() {
        return 0.0;
    }

    public double getArea() {
        return 0.0;
    }
}

class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    public double getArea() {
        return Math.PI * Math.pow(radius, 2);
    }
}

public class workshop4 {
    Run | Debug
    public static void main(String[] args) {
        Circle circle = new Circle(radius:3.670);
        System.out.println("Circle Perimeter: " + circle.getPerimeter());
        System.out.println("Circle Area: " + circle.getArea());
    }
}
```

```
Circle Perimeter: 23.059290077349083
Circle Area: 42.313797291935565
PS C:\Users\sigma\OneDrive\Desktop\java>
```

Super Keyword:

3. Extend the Animal and Dog example by adding a constructor to the Animal class that takes a name parameter. In the Dog class, use the super keyword to call the constructor of the Animal class. Create instances of Dog and demonstrate the use of the super keyword.

```
public class workshop4 {  
    Run | Debug  
    public static void main(String[] args) {  
        Dog dog1 = new Dog(name:"Buddy", breed:"Labrador");  
        dog1.display();  
    }  
}  
  
class Animal {  
    String name;  
  
    public Animal(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
    }  
}  
  
class Dog extends Animal {  
    String breed;  
  
    public Dog(String name, String breed) {  
        super(name);  
        this.breed = breed;  
    }  
  
    @Override  
    void display() {  
        super.display();  
        System.out.println("Breed: " + breed);  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> javac workshop4.java
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4
Name: Buddy
Breed: Labrador
```

Protected Keyword:

4. Create a class Person with a protected attribute **address**. Extend it with a subclass Employee that adds a **department** attribute. Demonstrate how the protected keyword allows access to the address property in the Employee subclass.

```
public class workshop4 {  
    Run | Debug  
    public static void main(String[] args) {  
        Employee employee = new Employee(name:"bibek", address:"ktm", department:"ca");  
        employee.display();  
    }  
}  
  
class Person {  
    protected String address;  
  
    public Person(String address) {  
        this.address = address;  
    }  
}  
  
class Employee extends Person {  
    private String department;  
  
    public Employee(String name, String address, String department) {  
        super(address);  
        this.department = department;  
    }  
  
    public void display() {  
        System.out.println("Name: " + address);  
        System.out.println("Department: " + department);  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> javac workshop4.java  
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4  
Name: ktm  
Department: ca  
PS C:\Users\sigma\OneDrive\Desktop\java> █
```

Access Modifiers and Inheritance:

5. Create class Parent with a private variable, a protected variable, and a public variable. Create a subclass Child

and demonstrate how each type of variable is accessed (or not accessed) within the subclass.

```
class workshop4 {  
    Run | Debug  
    public static void main(String[] args) {  
        Child child = new Child();  
        child.display();  
    }  
}  
  
class Parent {  
    private int privateVar = 10;  
    protected int protectedVar = 20;  
    public int publicVar = 30;  
}  
  
class Child extends Parent {  
    public void display() {  
        System.out.println("Protected variable in parent: " + protectedVar);  
        System.out.println("Public variable in parent: " + publicVar);  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4  
Protected variable in parent: 20  
Public variable in parent: 30  
PS C:\Users\sigma\OneDrive\Desktop\java> █
```

Final Classes and Methods:

6. Create a final class FinalClass. Attempt to extend it with another class and observe the compiler error. Also,

create a final method within a class and try to override it in a subclass.

```
class workshop4 {  
    Run | Debug  
    public static void main(String[] args) {  
        FinalClass finalObj = new FinalClass();  
        finalObj.display();  
    }  
}  
  
final class FinalClass {  
    final void display() {  
        System.out.println(x:"This is a final method");  
    }  
}
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> javac workshop4.java  
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4  
This is a final method
```

Method overloading

7. Create a class ,'Calculator' that should be able to perform addition operations for both integers and doubles. Implement the following steps:

a. Create an instance of the Calculator class.

- b. Use the add method to add two integers (e.g., 5 and 8) and display the result.
- c. Call the add method with three integers (e.g., 10, 15, and 20) and display the result.
- d. Use the add method to add two doubles (e.g., 3.5 and 2.7) and display the result.
- e. Call the add method with three doubles (e.g., 1.1, 2.2, and 3.3) and display the result.

```
class workshop4 {  
    Run | Debug  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
  
        // Adding two integers  
        int result1 = calc.add(num1:5, num2:8);  
        System.out.println("Result of adding two integers: " + result1);  
  
        // Adding three integers  
        int result2 = calc.add(num1:10, num2:15, num3:20);  
        System.out.println("Result of adding three integers: " + result2);  
  
        // Adding two doubles  
        double result3 = calc.add(num1:3.5, num2:2.7);  
        System.out.println("Result of adding two doubles: " + result3);  
  
        // Adding three doubles  
        double result4 = calc.add(num1:1.1, num2:2.2, num3:3.3);  
        System.out.println("Result of adding three doubles: " + result4);  
    }  
}  
  
class Calculator {  
    int add(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    int add(int num1, int num2, int num3) {  
        return num1 + num2 + num3;  
    }  
}
```

```
class Calculator {  
    int add(int num1, int num2) {  
        return num1 + num2;  
    }  
  
    int add(int num1, int num2, int num3) {  
        return num1 + num2 + num3;  
    }  
  
    double add(double num1, double num2) {  
        return num1 + num2;  
    }  
  
    double add(double num1, double num2, double num3) {  
        return num1 + num2 + num3;  
    }  
}
```

THIS IS A FINAL METHOD

PS C:\Users\sigma\OneDrive\Desktop\java> javac workshop4.java

PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4

Result of adding two integers: 13

Result of adding three integers: 45

Result of adding two doubles: 6.2

Result of adding three doubles: 6.6

Section -2

Case Study

Output: If User enters 2

Thank you for using the Application.

Criteria Online ordering has enabled many restaurants to manage their peak business hours very effectively. Thanks to online ordering, many people manage to prevent the painful experience of wasting time in a long queue.

AZA is one of the biggest restaurant chains in the United Kingdom. They decided to offer their customers a convenient and contactless mobile ordering solution in response to the growing business need and COVID restrictions.

As part of the development team in the SAS software solutions, you are required to design and develop the mobile ordering program.

Before the application release deadline, you are required to submit the following deliverables:

1. Java program for the two tasks as per the description that follows in this document.

Program Description:

In this program you have to create a signup process for the mobile ordering application for a restaurant. The restaurant has a variety of cuisines to offer their customers.

When the program starts, the user is given the following options:

1. Sign up
2. Quit Application

Output:

Please enter 1 for Sign up.
Please enter 2 for Quit.

Your program should keep running and enable multiple users to sign up until the quit application option is selected.

To start using the mobile app, users should sign up for an account. You are required to create a Java program for the signup process. The users will be asked to enter their full name, contact number, date of birth, password, and password confirmation.

1. The signup process must not be successful until:
 - a. The full name is enter. The length must be greater than four.
 - b. The mobile number has 10 digits starting with 0.
 - c. The Password must initiate with alphabets followed by either one of @, & and ending with numeric.
(For Example: John@0125 or John&25) .
 - d. The password confirmation matches the initial entered password.
 - e. The DOB is in the format DD/MM/YYYY or MM/DD/YYYY.
 - f. The user is at least 21 years old. The age should be calculated based on the year entered in the DOB (Only consider year).
2. If any of the above-mentioned conditions is not fulfilled; the sign-up process should fail, and a descriptive message should be displayed

for the user explaining what has gone wrong and providing hints on the correct expected input. The program should keep asking the user to re-enter his details as long as one or more of the input fields are not correctly entered. If all fields are entered successfully, the program should stop asking the user to re-enter his details and display a message that the signup process has been completed successfully.

3. If any field is entered incorrectly, some examples of sample outputs are given below.

Output 1:

You have entered the Date of Birth in invalid format.
Please start again.

Output 2:

Your passwords are not matching.
Please start again.

4. If all of the above-mentioned conditions are successful, the user data is saved in appropriate data structure (Hint: Arrays can be used) to enable data checks during the login process in future builds.

Output 1:

Please enter your full name: Sam Jahn

Please enter your mobile number (username): 0445544455

Please enter your password: John@21

Please confirm your password: John@21

Please enter your Date of Birth #DD/MM/YYYY (No space):
21/01/1984

You have successfully signed up.

Please enter 1 for Sign up.
Please enter 2 for Quit.

```
import java.util.Scanner;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class workshop4 {
    Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice = 0;

        while (choice != 2) {
            System.out.println(x:"Please enter 1 for Sign up.");
            System.out.println(x:"Please enter 2 for Quit.");
            choice = scanner.nextInt();

            switch (choice) {
                case 1:
                    signUp();
                    break;
                case 2:
                    System.out.println(x:"Thank you for using the Application.");
                    break;
                default:
                    System.out.println(x:"Invalid choice. Please enter 1 or 2.");
                    break;
            }
        }

        scanner.close();
    }

    public static void signUp() {
        Scanner scanner = new Scanner(System.in);
        System.out.println(x:"Sign Up");
        System.out.print(s:"Enter your full name: "):
```

```

System.out.print(s:"Enter your full name: ");
String fullName = scanner.nextLine();
if (fullName.length() <= 4) {
    System.out.println(x:"Full name must be longer than four characters.");
    return;
}

System.out.print(s:"Enter your contact number: ");
String contactNumber = scanner.nextLine();
if (!isValidContactNumber(contactNumber)) {
    System.out.println(x:"Invalid contact number format. It must have 10 digits starting with 0.");
    return;
}

System.out.print(s:"Enter your date of birth (DD/MM/YYYY): ");
String dobString = scanner.nextLine();
if (!isValidDOB(dobString)) {
    System.out.println(x:"Invalid date of birth format. It must be in the format DD/MM/YYYY or MM/DD/YYYY.");
    return;
}

LocalDate dob = LocalDate.parse(dobString, DateTimeFormatter.ofPattern(pattern:"dd/MM/yyyy"));
if (!isOver21(dob)) {
    System.out.println(x:"You must be at least 21 years old to sign up.");
    return;
}

System.out.print(s:"Enter your password: ");
String password = scanner.nextLine();
System.out.print(s:"Confirm your password: ");
String confirmPassword = scanner.nextLine();
if (!isValidPassword(password, confirmPassword)) {
    System.out.println(x:"Invalid password format. Password must start with alphabets, followed by either '@' or '&', and end with a numeric");
    return;
}

```

```

}

System.out.println(x:"Signup successful!");

// Additional logic for saving user information to a database or file can be added here

scanner.close();
}

public static boolean isValidContactNumber(String contactNumber) {
    return contactNumber.matches(regex:"^0\\d{9}$");
}

public static boolean isValidDOB(String dobString) {
    return dobString.matches(regex:"^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\\d{4}$") || dobString.matches(regex:"^(0[1-9]|1[0-2])/(0[1-9]|[12][0-9]|3[01])/\\d{4}$");
}

public static boolean isOver21(LocalDate dob) {
    LocalDate today = LocalDate.now();
    return today.minusYears(yearsToSubtract:21).isAfter(dob);
}

public static boolean isValidPassword(String password, String confirmPassword) {
    return password.matches(regex:"^[a-zA-Z]+[@&]\\d+$") && password.equals(confirmPassword);
}
}

```

```
1
Sign Up
Enter your full name: Bibek
Enter your contact number: 0987654432
Enter your date of birth (DD/MM/YYYY): 09/09/1990
Enter your password: John@123
Confirm your password: John@123
Signup successful!
Please enter 1 for Sign up.
Please enter 2 for Quit.
Exception in thread "main" java.util.NoSuchElementException
    at java.base/java.util.Scanner.throwFor(Scanner.java:945)
    at java.base/java.util.Scanner.next(Scanner.java:1602)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2267)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2221)
    at workshop4.main(workshop4.java:282)
PS C:\Users\sigma\OneDrive\Desktop\java>
```

```
PS C:\Users\sigma\OneDrive\Desktop\java> java workshop4
Please enter 1 for Sign up.
Please enter 2 for Quit.
2
Thank you for using the Application.
PS C:\Users\sigma\OneDrive\Desktop\java>
```